



DICOM Hub

Multi-Hospital DICOM Sharing Platform

A full-stack Medical Imaging Portal for secure cross-facility sharing of radiological studies, patient records, and imaging requests.

Project Name	DICOM Hub — Medical Imaging Portal
Category	Healthcare / Medical Imaging (PACS-like)
Version	1.0.0
Platform	Web (responsive, desktop-first)
Primary Backend	Appwrite Cloud (Database, Auth, Storage)

1. Introduction

DICOM Hub is a comprehensive web-based medical imaging platform that enables hospitals to share DICOM studies, manage patient-centric records, and coordinate inter-hospital imaging requests. The system provides a professional, clinical-grade interface inspired by leading healthcare software (Epic MyChart and Philips IntelliSpace Portal) while delivering real-time analytics and a scalable cloud backend.

The application bridges a modern React front end with an Appwrite cloud backend for database, authentication, and file storage, and ships with an Express REST API layer for standalone or server-rendered deployment.

1.1 Problem Statement

Hospitals frequently need to share radiological studies between facilities for second opinions, referrals, and continuity of care. Manual transfer via media or email is slow, insecure, and unmanageable at scale. There is a need for a secure, web-accessible platform that centralizes DICOM management, tracks patient records, and formalizes cross-hospital imaging requests with approval workflows.

1.2 Objectives

- Centralize DICOM study upload, archive, and retrieval across connected hospitals.
- Provide complete patient-centric medical records with imaging attachments.
- Enable structured inter-hospital imaging requests with status tracking and approval workflows.
- Deliver operational analytics and system-health monitoring for administrators.
- Establish an architecture ready for HIPAA-aligned deployment and PACS integration.

1.3 Scope

The current release covers: patient management, DICOM upload and archive browsing, recently uploaded tracking, hospital imaging requests, a DICOM viewer interface, user settings, connection diagnostics, and dashboard analytics. DICOM frame rendering is designed for integration with Cornerstone.js / OHIF; the viewer interface and all required libraries are already wired into the codebase.

2. Technology Stack

2.1 Frontend

Layer	Technology	Purpose
-------	------------	---------

Framework	React 18.3 + TypeScript 5.6	Component-based SPA with static typing
Build tool	Vite 5.4	Fast dev server with HMR and production bundling
Styling	Tailwind CSS 3.4	Utility-first responsive styling
UI kit	shadcn/ui (Radix UI)	Accessible, themeable component primitives
Routing	Wouter	Lightweight client-side routing
Data fetching	TanStack React Query 5	Server-state caching and mutation handling
Charts	Recharts	Analytics visualizations
Forms	react-hook-form + Zod	Validated form handling

2.2 DICOM / Medical Imaging

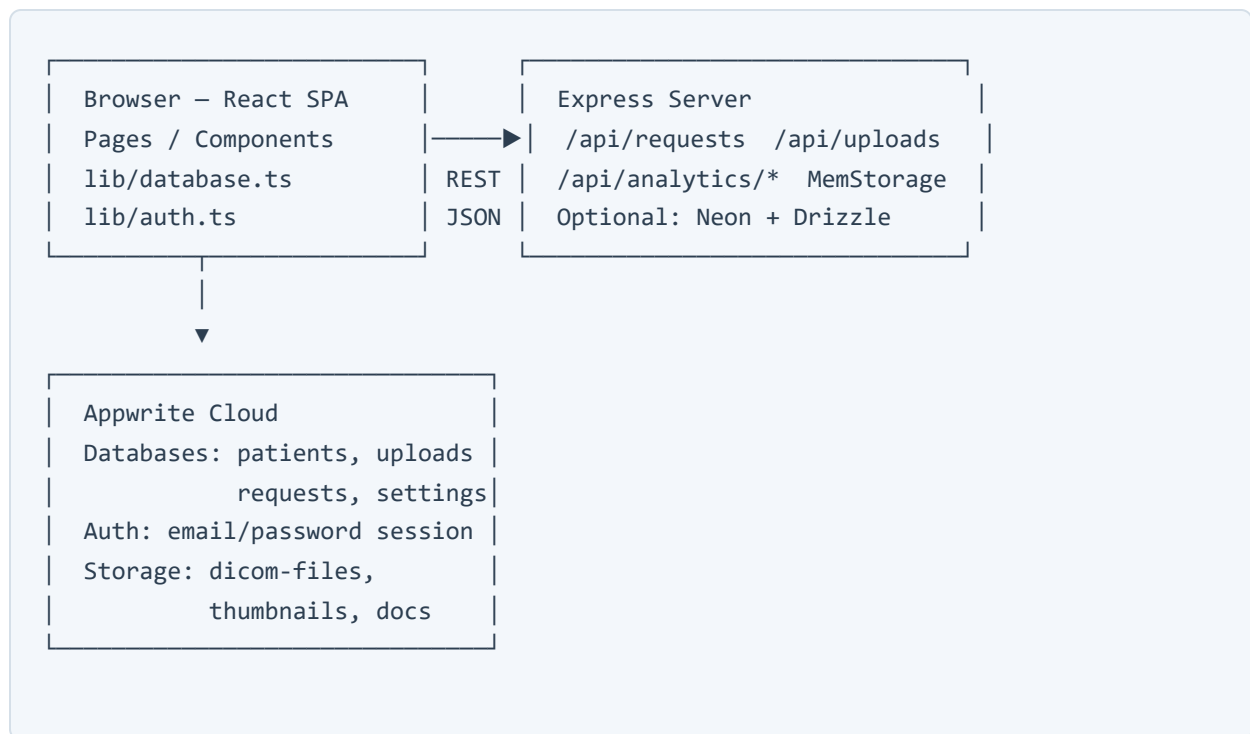
Library	Role
dicom-parser	Parses DICOM metadata from .dcm files
cornerstone-core	High-performance medical image rendering engine
cornerstone-tools	Viewer tools — window/level, zoom, pan, measurement
cornerstone-math	Geometric utilities for measurements
cornerstone-wado-image-loader	WADO/HTTP DICOM image loading

2.3 Backend & Cloud

Layer	Technology	Role
API server	Express 4.21 (TypeScript via tsx)	REST endpoints under <code>/api/</code>
Data layer	Appwrite Cloud (Databases, Auth, Storage)	Primary persistence, sessions, file buckets
Optional DB	Neon PostgreSQL + Drizzle ORM	SQL schema and migrations via drizzle-kit
Demo storage	In-memory MemStorage (seeded)	Standalone runnable backend with sample data

3. System Architecture

The system follows a modular client-server architecture. The React SPA communicates with Appwrite Cloud through a typed service layer for all primary data operations, while an Express server provides REST APIs, development-time Vite middleware, and production static hosting.



3.1 Key Design Decisions

- **Appwrite-first data access:** all pages consume live data through one centralized service layer (`client/src/lib/database.ts`), eliminating hardcoded endpoints.
- **Type safety:** full TypeScript interfaces mirror the Appwrite document schemas and the Drizzle SQL schema.
- **Resilience:** comprehensive try/catch with user-facing toasts, loading skeletons, and pagination throughout.
- **Extensibility:** Cornerstone.js dependencies and a viewer page are pre-wired for full DICOM rendering.

4. Feature Modules

9 APPLICATION PAGES	5 APPWRITE COLLECTIONS	4+ IMAGING MODALITIES	13 REST API ROUTES
----------------------------------	-------------------------------------	------------------------------------	------------------------------

Module	Route	Capabilities
--------	-------	--------------

Dashboard	/	KPI cards, recent uploads, modality distribution, recent activity, hospital rankings, critical alerts, system health panel.
Patients	/patients	Patient CRUD, full-text search, rich profile dialogs, medical file uploads, history/allergies/medications/insurance.
DICOM Upload	/upload	Drag-and-drop upload zone, metadata capture (patient, modality, body part, date, notes), compliance guidelines.
Recently Uploaded	/recently-uploaded	Paginated table, search, modality/status filters, detail card, guarded deletion.
DICOM Archive	/dicom-archive	Studies grouped by patient, generated patient profiles, date-range and modality filters, share/download actions.
Hospital Requests	/requests	Request creation with file attachment (website or PC), status tabs, approve/reject workflow, pagination.
DICOM Viewer	/viewer/:id	Study metadata view, download action, Cornerstone-ready viewer surface.
Settings	/settings	Profile, notifications, system defaults, security controls, live Appwrite connection testing.
Connection Test	/connection-test	Diagnostic validation of endpoint, project, database, and collections.

5. Data Model

5.1 Appwrite Collections

Collection	Key Fields
patients	patientId, name, dateOfBirth, gender, contactInfo, emergencyContact, medicalHistory, allergies, currentMedications, insuranceInfo
uploads	fileName, patientName, patientId, studyType, modality, uploaderName, uploaderHospital, fileSize, status, description, fileUrl, thumbnailUrl
requests	requestId, patientName, patientId, requestingPhysician, department, hospital, urgency, studyType, modality, clinicalInfo, status, scheduledDate, completedDate, notes
users	User accounts managed by Appwrite Auth
settings	userId, theme, notifications, systemSettings

5.2 Storage Buckets

- `dicom-files` — DICOM study files
- `thumbnails` — image thumbnails
- `documents` — reports and documents

5.3 Optional SQL Schema (Drizzle)

A PostgreSQL schema (`shared/schema.ts`) models `users` , `requests` , and `uploads` tables with UUID primary keys, timestamps, and status/priority enums. Drizzle Kit pushes this schema with `npm run db:push` .

6. REST API (Express)

Method	Endpoint	Description
GET	<code>/api/requests</code>	List requests with status filter + pagination
GET	<code>/api/requests/:id</code>	Fetch single request
POST	<code>/api/requests</code>	Create request (Zod-validated)
PATCH	<code>/api/requests/:id</code>	Update request status
DELETE	<code>/api/requests/:id</code>	Delete request
GET	<code>/api/uploads</code>	List uploads (newest first, paginated)
GET	<code>/api/uploads/:id</code>	Fetch single upload
POST	<code>/api/uploads</code>	Create upload record
DELETE	<code>/api/uploads/:id</code>	Delete upload
GET	<code>/api/analytics/stats</code>	Aggregate KPIs
GET	<code>/api/analytics/modality-stats</code>	Modality counts and percentages
GET	<code>/api/analytics/hospital-stats</code>	Per-hospital upload/request counts
GET	<code>/api/analytics/recent-activity</code>	Recent activity timeline

7. Installation & Deployment

7.1 Prerequisites

- Node.js $\geq$ 18 (tested on 20.x / 24.x)
- npm or yarn package manager

- An Appwrite Cloud project (endpoint + project ID)

7.2 Commands

```
npm install          # install dependencies
npm run dev          # start dev server (HMR + API) on PORT
npm run check        # TypeScript type-check
npm run build && npm start # production build and run

# Appwrite setup & verification
npm run setup:appwrite # create collections, buckets, indexes
npm run seed:appwrite  # seed realistic demo data
npm run update:permissions # fix collection permissions
npm run test:integration # end-to-end integration test
npm run appwrite:check  # CLI connectivity check
```

7.3 Environment Configuration

```
VITE_APPWRITE_PROJECT_ID = "your-project-id"
VITE_APPWRITE_ENDPOINT   = "https://<region>.cloud.appwrite.io/v1"
PORT                     = 3000
# Optional (Express/Drizzle persistence)
DATABASE_URL             = "postgres://user:pass@host/db"
```

8. Security & Compliance

- **Authentication:** Appwrite email/password sessions; user preferences store role, department, and hospital.
- **Validation:** all Express routes validate payloads with Zod before persistence.
- **Secret management:** API keys and credentials are environment-based and git-ignored.
- **Compliance foundation:** consent documentation, HIPAA-aligned upload guidelines, and audit-ready workflows.
- **Transport:** HTTPS endpoints for Appwrite; TLS expected at the production edge.

9. Future Work

- Render live DICOM frames in the viewer via Cornerstone.js / OHIF.
- Persist the Express layer to PostgreSQL (schema and Drizzle config in place).
- Role-based access control for radiologists, referring physicians, and admins.

- Real-time updates via Appwrite subscriptions / WebSockets.
- Email/push notifications for request approvals and upload completions.
- Direct PACS integration via DICOM C-STORE / WADO-RS.

10. Conclusion

DICOM Hub delivers a production-ready medical imaging portal that connects hospitals and radiologists through secure, centralized DICOM management. With a clinical-grade interface, a typed cloud data layer, structured request workflows, and a Cornerstone-ready viewer, the platform provides a strong foundation for real-world hospital deployment and PACS integration.